带 AGV 运输与有限缓冲阻塞的柔性作业车间联合调度：建模、死锁规避与学习型构造策略

（作者）

2026-11-04（草稿）

草稿说明 本文是大学生创新训练项目（大创）论文初稿，对应仓库代码 `src/fjsp_lab/logistics/`。文中所有数值都取自 `experiments/results/` 下已归档的 6 个实验目录：`l1_benchmark`、`l1_decoupled`、`l1_large`、`l1_search`、`l2_formal_train`、`l2_formal_test`。每个目录的 `manifest.json` 记录了对应的代码提交与输入文件哈希。投稿前需要完成：

- 参考文献中标注 [**待核对**] 或 [**待补全**] 的条目，按原文核对或补全作者、题目和卷期；只读过摘要的文献需读全文确认文中的表述；
- 投稿前再做一次数据库检索（WoS/Scopus）和引文追溯，确认第 2 节的定位；
- 按目标期刊或会议的模板调整格式与篇幅。

摘要

在柔性作业车间中，机器加工与自动导引车（AGV）运输相互耦合；当机器前的缓冲容量有限、加工完的工件必须等 AGV 取走才释放机器（阻塞）时，非延迟的派工规则还会陷入死锁。已有的柔性作业车间运输调度（FJSPT）研究普遍假设缓冲无限，而考虑有限缓冲的学习型方法又不含运输。本文在 FJSPT 的时间框架下（AGV 有位置，空驶与载货行驶分别计时，工件从装卸站出发）引入有限输入缓冲、加工后阻塞和派车预留，建立以最大完工时间为目标的约束模型，给出 CP-SAT 参考模型与独立的合法性检查器，并在公开基准上复现了全部 10 个参考解。在此基础上提出前瞻式死锁规避：每个决策先在仿真副本上用保守策略推演到结束，推演成功才执行。它既用来包装派工规则，也作为强化学习的动作掩码。学习型策略为每个“工序—机器—AGV”候选动作打分，先模仿最好的规避规则，再用带共享基线的 REINFORCE 微调。实验表明：（1）无缓冲加阻塞使小算例的最优值平均上升 6.8%–12.8%，使大算例的启发式解上升约 40%–50%；（2）先排机器、再排车辆的解耦方法比联合优化平均差 5%–10%，无缓冲时升至 17%–20%，且有 2–3 个算例的机器方案无法执行；（3）在预先登记的检验中，学习型策略在 3 个训练种子上都显著优于规则和同等时间的模拟退火；在比训练算例大得多的 Brandimarte 算例上，每例采样 32 次取最好，比逐例最好规则好 16.3%，比大预算模拟退火好 9.5%；但在小算例上距证得最优仍有约 12.5% 的差距。

关键词：柔性作业车间调度；AGV；有限缓冲；阻塞；死锁规避；强化学习

Abstract. We study flexible job shop scheduling with AGV transport (FJSPT) under finite input buffers and blocking after processing, where non-delay dispatching may deadlock. We give a constraint model with explicit empty and loaded travel, a CP-SAT reference model and an independent feasibility checker that replays all ten published benchmark solutions. A lookahead deadlock guard, which accepts a decision only if a conservative rollout on a copy of the simulator finishes every job, wraps dispatching rules and serves as the action mask of a learned per-candidate scoring policy trained by imitation and REINFORCE with a shared baseline. Blocking without buffers raises small-instance optima by 6.8%–12.8% and large-instance heuristic makespans by about 40%–50%; a decoupled machine-then-vehicle approach is 5%–10% worse than joint optimization, 17%–20% without buffers. In pre-registered tests over three training seeds the policy beats the rules and equal-time simulated annealing; on the larger Brandimarte instances 32 samples improve on the best rule by 16.3% and on long-run annealing by 9.5%, while on small instances the gap to proven optima is still about 12.5%.

1 引言

柔性作业车间调度问题（FJSP）要为每道工序选择机器并确定加工顺序，是生产调度中研究最多的问题之一 [1]。在实际车间里，工件要由 AGV 在装卸站与机器之间搬运。运输时间不可忽略，AGV 卸货后还要空驶到下一个取货点，空驶时间取决于前后两次任务的位置，所以机器调度和车辆调度必须一起考虑。这类问题称为带运输资源的柔性作业车间调度（FJSPT），从 Bilge 与 Ulusoy 的工作 [2] 开始已有大量研究，近年的精确与元启发式方法见文献 [4, 5]。

现有 FJSPT 研究，尤其是深度强化学习方法 [9, 10, 11]，普遍假设机器前后的缓冲无限：工件随时可以送到机器前排队，加工完立即离开机器。实际车间的工位空间有限，常见的情形是机器前只有一两个等待位，加工完的工件留在机器上，直到 AGV 取走才释放机器。这时会出现两个新问题：

1. **缓冲和阻塞改变了问题的结构。**机器被成品占着时不能开工，AGV 来晚了会直接延误下游；机器分配、加工顺序和运输顺序之间的耦合更强。
2. **死锁。**机器 A 上的成品要去机器 B，B 上的成品要去 A，而两台机器前都没有空位，两个工件就会永远等下去。按非延迟规则派工时，这种情形并不罕见：本文的大算例上，缓冲容量不超过 2 时普通规则经常死锁，无缓冲时几乎全部死锁（第 5.5 节）。

有限缓冲与死锁在柔性制造系统的 Petri 网研究中有很长的历史，Hu 等 [12] 用 Petri 网加深度 Q 网络处理了有限资源与死锁避免；Zhang 等 [13] 用强化学习求解有限缓冲的柔性作业车间，但不含运输；也有中文期刊论文在流水车间类问题中把运输、阻塞与强化学习结合起来 [14, 15]。据我们的检索（见第 2 节），同时具备“柔性路线、AGV 显式行驶（含空驶）、有限缓冲与阻塞、死锁处理、学习型方法”的工作尚未见报道。柔性路线使机器选择和运输目的地都在决策时才确定，这与固定路线的流水车间不同。

本文的主要工作如下：

1. **模型与参考解。**在 FJSPT 的时间框架下引入有限输入缓冲、加工后阻塞和派车预留，给出约束模型、CP-SAT 参考模型与独立的合法性检查器。按基准约定，公开的 10 个参考解全部复现；CP-SAT 在 9 个算例上证得与参考值相同的最优值，在 fjsp7 上找到 108，优于公开的 112。
2. **缓冲与协同的定量分析。**在小算例上用证得最优值、在大算例上用规则与模拟退火，量化

阻塞与缓冲容量的影响；并在同样的预算下比较联合优化与“先排机器、再排车辆”的解耦优化，得到解耦代价随缓冲变紧的变化。

3. **前瞻式死锁规避与学习型策略。**提出基于保守推演的死锁规避，把它同时用于规则基线和强化学习的动作掩码；学习型策略对“工序—机器—AGV”联合动作逐个打分，只有 704 个参数，先模仿再用 REINFORCE 微调。
4. **预先登记的评价。**训练前固定假设、预算与比较口径，3 个训练种子分别报告；测试集只使用一次。

第 2 节回顾相关工作；第 3 节给出问题与模型；第 4 节介绍求解方法；第 5 节报告实验；第 6 节总结并讨论局限。

2 相关工作

机器与运输联合调度。 Bilge 与 Ulusoy[2] 研究了柔性制造系统中机器与物料搬运系统的同时调度，其布局和算例被后来的研究反复使用。Deroussi 与 Norre[3] 把它推广到柔性作业车间，给出了本文使用的 10 个小算例。Homayouni 与 Fontes[4] 提出了生产与运输联合调度的模型和算法，并给出了大算例所用的布局；Berterottière 等 [5] 给出了带运输资源的柔性作业车间的模型与方法，并公开了参考解。Zhuang 等 [7] 用混合整数规划求解分区柔性制造系统中机器与 AGV 的同时调度，其中加工区不设缓冲；Moinuddin 等 [8] 用约束规划处理异构运输机器人与分区，并用路径预订避免碰撞。这些工作或假设缓冲充足，或采用精确与元启发式方法，没有使用学习型方法。

有限缓冲、阻塞与死锁。 在有限缓冲和阻塞下，调度必须避免循环等待。Petri 网是柔性制造系统死锁控制的主流工具，Hu 等 [12] 在 Petri 网上用图卷积与深度 Q 网络做动态调度，并处理死锁避免。中文文献中也有柔性作业车间在有限缓冲与 AGV 下的遗传算法研究，以及用时序 Petri 网求解机器人柔性作业车间无死锁调度的工作（见查新报告）。本文不建 Petri 网，而是直接在事件仿真器上做前瞻推演，这样可以沿用 FJSPT 的时间模型和公开基准，与该领域的精确解直接比较。

柔性作业车间的深度强化学习。 Zhang 等 [16] 在作业车间中用图神经网络和策略梯度学习派工，Song 等 [17] 把这一思路推广到柔性作业车间；这一线的方法通常把“工序—机器”对作为动作。对于带运输的问题，Link 等 [9] 比较了联合学习与模块化学习的“协调差距”，但其模型是固定路线的作业车间且缓冲无限；HA-DQN[10] 和 Wang 等 [11] 研究了带运输约束的柔性作业车间，同样假设缓冲无限。有限缓冲的学习型方法 [13] 没有运输；运输、阻塞与强化学习的结合见于流水车间类问题 [14, 15]。训练方法上，本文采用 REINFORCE[18]，基线取同一实例多次采样的平均值，与 Kool 等 [19]、Kwon 等 [20] 的共享基线思路相同。

本文的定位。 表 1对比了最接近的工作。本文的贡献是组合式的：把学习型构造策略推广到带 AGV 显式行驶、有限缓冲与阻塞的柔性作业车间，并用前瞻推演保证死锁规避；同时给出缓冲容量与联合/解耦差距之间关系的定量结果。

表 1: 与最接近工作的对比（✓ 有，× 无，？ 未能核实）

文献	柔性路线	AGV 显式行驶	有限缓冲/阻塞	死锁处理	方法
Hu 等 2020[12]	✓	?	✓/ ?	✓	Petri 网 + DQN
Zhang 等 2026[13]	✓	×	✓/ ×	×	PPO + 异构图
Link 等 2026[9]	×	✓	×/ ×	×	MAPPO
HA-DQN 2025[10]	✓	✓	×/ ×	×	DQN
Zhuang 等 2025[7]	✓	✓	无缓冲 / ✓	假设不发生	MILP
Moinuddin 等 2026[8]	✓	✓	×/ ×	路径预订	CP
本文	✓	✓	✓/ ✓	前瞻推演	模仿 + REINFORCE

3 问题描述与数学模型

3.1 问题描述与假设

车间有机器集合 $\mathcal{M} = \{1, \dots, m\}$ 和一个装卸站 ℓ，工件集合 $\mathcal{J}$。工件 j 有 n_j 道按顺序加工的工序，工序 (j, o) 可在机器集合 $\mathcal{M}_{jo} \subseteq \mathcal{M}$ 中任选一台，在机器 i 上的加工时间为 p_{joi}。V 台同质、单载 AGV 组成集合 $\mathcal{V}$，0 时刻都停在 ℓ。地点之间的行驶时间为 τ_{ab}（$a, b \in \mathcal{M} \cup \{\ell\}$），空驶与载货行驶使用同一矩阵，对角线为 0；每次载货行驶另加装卸时间 $h \geq 0$。机器 i 前有容量为 $B_i \in \mathbb{Z}_{\geq 0} \cup \{\infty\}$ 的输入缓冲。其余假设：

- 工件 0 时刻都在 ℓ；每道工序加工一次，不可中断；运输一旦开始不可中断。
- 同一工件在同一台机器上连续加工的工序之间不需要运输。
- **阻塞**：加工完的工件留在机器上，直到 AGV 取走才释放机器；机器被占着时不能开始其他工件的加工。
- **工位容量**：机器 i 的工位共 $1 + B_i$ 个位置（机器本身加缓冲）。派车时即在目的工位预留一个位置，工件被取走（或最后一道工序结束且不需运走）时释放。ℓ 容量无限。
- 两个开关：阻塞开关 $\beta \in \{0, 1\}$，关闭时工件加工完立即进入容量无限的输出区，不占机器也不占工位；送回开关 $R \in \{0, 1\}$，打开时工件最后要运回 ℓ。FJSPT 基准算例的约定是 $\beta = 0$、$R = 0$、$B_i = \infty$、$h = 0$。
- 不考虑路径冲突、拥堵和充电。

目标是最小化最大完工时间 $C_{\max}$。$R = 1$ 时工件完工时刻为运回 ℓ 的送达时刻，否则为最后一道工序的结束时刻。

3.2 约束模型

工件 j 的运输以“送往的工序号”o 标识，$o = 0, \dots, n_j - 1$，$R = 1$ 时另有送回 ℓ 的运输 $o = n_j$。记 μ_{jo} 为工序 (j, o) 所选机器，运输 (j, o) 的起点 $a_{jo} = \mu_{j,o-1}$（$o = 0$ 时为 ℓ），终点 $b_{jo} = \mu_{jo}$（$o = n_j$ 时为 ℓ）。决策变量：

- $x_{joi} \in \{0, 1\}$：工序 (j, o) 是否在机器 i 上加工；S_{jo}, C_{jo}：开工与完工时刻；
- $y_{jo} \in \{0, 1\}$：运输 (j, o) 是否需要；$z_{jov} \in \{0, 1\}$：是否由 AGV v 执行；
- $D_{jo} \leq P_{jo} \leq Q_{jo}$：派车、取货、送达时刻。

加工：

$$\sum_{i\in\mathcal{M}_{jo}} x_{joi}=1, \qquad C_{jo}=S_{jo}+\sum_i p_{joi}x_{joi}, \tag{1}$$

$$S_{jo}\ge C_{j,o-1} \quad (o\ge 1). \tag{2}$$

运输：$y_{j0}=1$，$R=1$ 时 $y_{jn_j}=1$；$1\le o<n_j$ 时 $y_{jo}=1$ 当且仅当 $\mu_{jo}\neq\mu_{j,o-1}$。

$$Q_{jo}=P_{jo}+y_{jo}\,(\tau_{a_{jo}b_{jo}}+h), \tag{3}$$

$$P_{jo}\ge C_{j,o-1}\ (o\ge 1), \qquad S_{jo}\ge Q_{jo}\ (o<n_j), \tag{4}$$

$$y_{jo}=0\ \Rightarrow\ D_{jo}=P_{jo}=C_{j,o-1}. \tag{5}$$

车辆：$\sum_v z_{jov}=y_{jo}$。同一 AGV 的运输串成一条路径：若运输 k' 紧接在 k 之后，则

$$D_{k'}\ge Q_k, \qquad P_{k'}\ge D_{k'}+\tau_{b_k a_{k'}}; \tag{6}$$

若 k 是该车的第一次运输，则 $D_k\ge 0$、$P_k\ge D_k+\tau_{\ell a_k}$。

阻塞：工序 (j,o) 对机器的占用区间为 $[S_{jo},F_{jo})$，其中

$$F_{jo}=\begin{cases} P_{j,o+1}, & \beta=1 \text{ 且} y_{j,o+1}=1,\\ S_{j,o+1}, & y_{j,o+1}=0\text{（工件留在本机加工下一道工序）},\\ C_{jo}, & \beta=0,\ \text{或}o=n_j-1 \text{ 且} R=0. \end{cases} \tag{7}$$

同一机器上不同工序的占用区间互不重叠：若 $x_{joi}=x_{j'o'i}=1$，则 $F_{jo}\le S_{j'o'}$ 或 $F_{j'o'}\le S_{jo}$。$\beta=1$ 时这比“加工不重叠”更严格。

工位容量：工序 (j,o) 在其机器的工位上从 r_{jo} 起占用一个位置，其中 $y_{jo}=1$ 时 $r_{jo}=D_{jo}$（派车即预留），否则 $r_{jo}=S_{jo}$；释放时刻 $\beta=1$ 时同式 (7)，$\beta=0$ 时为 C_{jo}（留在本机时为 $S_{j,o+1}$）。对任意机器 i 和时刻 t，

$$\big|\{(j,o): x_{joi}=1,\ r_{jo}\le t<\text{释放时刻}\}\big|\le 1+B_i. \tag{8}$$

目标：

$$\min C_{\max}, \qquad C_{\max}\ge\begin{cases} Q_{jn_j}, & R=1,\\ C_{j,n_j-1}, & R=0. \end{cases} \tag{9}$$

所有时间取整数，区间左闭右开。式 (6) 使空驶时间依赖前后两次运输的位置，而位置又取决于机器选择，这是机器与车辆必须联合决策的根本原因。

命题 1. 对任意算例和任意 $B_i\ge 0$，可行解都存在。

证明. 构造串行调度：工件逐个处理，前一个工件完工（$R=1$ 时运回 ℓ）后才派出下一个；只用一台 AGV，每次运输都等上一道工序结束再派车，每道工序任选一台可选机器。任一时刻车间里最多一个工件，所以各工位最多被占一个位置（$\le 1+B_i$），机器占用区间不重叠，车辆路径的时刻也都满足式 (6)。 □

注 1. 取 $\tau\equiv 0$、$h=0$、$\beta=0$、$R=0$、$B_i=\infty$ 时，运输不占时间，车辆约束不再起作用，模型退化为 FJSP；作业车间问题已是 NP 难的 [21]，所以本问题也是 NP 难的。命题 1 说明死锁不是问题本身无解，而是派工方式造成的。

3.3 死锁

图 1 是 $B = 0$ 时的死锁：工件 1 在 M_1 上加工完，下一道工序只能在 M_2 上做；工件 2 在 M_2 上加工完，下一道工序只能在 M_1 上做。两台机器都被成品占着，没有空位可以预留，两个工件都无法派车。模型允许两台 AGV 在同一时刻分别取走两个工件并互换机器（取货时机器即空出），但逐个决策的仿真器做不到这种同时互换。非延迟规则只看当前，一旦走进这样的状态就无法退出。

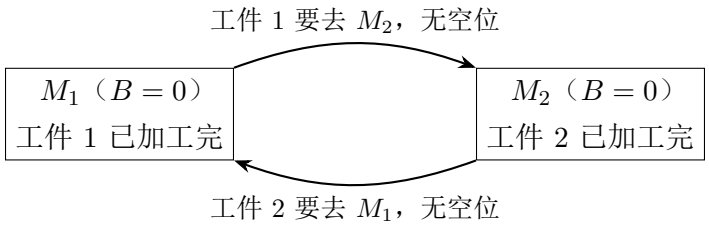


图 1: 阻塞且无缓冲时的循环等待

3.4 CP-SAT 参考模型与合法性检查器

小规模算例用 OR-Tools CP-SAT[22] 求解上述模型：每道工序在每台可选机器上一个可选区间，恰好一个出现；每台 AGV 的运输用一个 `AddCircuit` 约束串成回路（车场即 0 时刻的 ℓ）；阻塞用 `NoOverlap`，工位容量用 `Cumulative`；行驶时间依赖机器选择，用 `Element` 约束取值。求解器状态为 OPTIMAL 时才称最优，FEASIBLE 只作为限时可行解。

所有方法返回的调度都必须通过独立的合法性检查器。检查器把生产部分交给不改动的静态 FJSP 检查器（每道工序恰好一次、工艺顺序、可选机器、加工时长、加工不重叠），再逐条检查：所需运输恰好各一次且起讫正确；运输时长；取货不早于工序结束、开工不早于送达；车辆从 ℓ 出发、运输串联且空驶时间足够；阻塞下占用区间不重叠；工位占用不超过 $1 + B_i$；报告的 $C_{\max}$ 等于独立重算值。检查器只判定，不修复。

4 求解方法

4.1 事件驱动仿真与候选动作

规则和学习型策略都在一个事件驱动仿真器上构造调度。仿真器在“有事可做”的时刻（非延迟）停下来，给出候选动作集合。一个候选动作是三元组决策：把等待中的工件 j 用空闲 AGV v 送往下一道工序的某台可选机器 i（或送回 ℓ）；若工件所在机器也能加工下一道工序，还可以选择“原地留下”。目的工位已满的候选不出现。派车时锁定机器并预留目的工位；工件到达后在输入缓冲中按到达顺序等待，机器空闲时开工。同一时刻的事件按固定顺序处理（工序结束、取货、送达，然后开工，最后决策），保证结果可复现。

4.2 前瞻式死锁规避

对一个候选动作 c，在仿真器副本上执行 c，再用**保守推演策略**运行到结束：已进入车间的工件按联合最早完工（EFT）推进，车间清空之后才从 ℓ 放行下一个工件。推演完成则称 c **安**

全。推演成功即证明执行 c 后的状态仍能完成全部工件；推演失败不一定表示真的会死锁，所以这种规避是保守的。

规避包装器按策略的偏好顺序逐个检查候选，执行第一个安全的；都不安全则等待下一个事件。缓冲全部无限时不可能死锁（等待的工件总能被运走），不做检查，此时包装后的规则与原规则完全相同。在实验中，带规避的规则 420 次运行、学习型策略的全部训练与测试回合都没有出现死锁（第 5.5、5.6 节）。从推演的构造看，初始状态总是安全的，规避后每一步也保持安全，这是规避有效的直观原因；但同一时刻事件的处理顺序会让“执行后继续推演”与“原推演”略有不同，我们没有给出严格的完备性证明。

4.3 规则基线

联合规则 EFT 在全部“工序—机器—AGV”候选中选预计完工时刻最早者，同时考虑空驶、载货行驶和目的机器的排队。解耦规则先按 FIFO、SPT 或 MWKR 选工件，并在不计运输的前提下为它选机器，再派最近的空闲车辆（记为 +NV），对应“先排生产、再派车辆”的做法。下文的“规避规则”指加了前瞻规避的 EFT、FIFO+NV、SPT+NV、MWKR+NV，“逐例最好规则”指这 4 条中在该实例上 $C_{\max}$ 最小者。

4.4 学习型构造策略

决策过程。 状态为仿真器在决策时刻的全部信息，动作为一个候选，只在结束时给一次回报 $-C_{\max}/S$，其中尺度 $S=\sum_{j,o}\min_i p_{joi}/m$。

候选特征与打分。 每个候选用 20 维特征描述（表 2），时间量都除以 S。EFT、SPT、MWKR 和最近车辆等规则用到的信息都能由这些特征表示。打分函数为两层感知机

$$f(\boldsymbol{x})=\boldsymbol{w}_2^{\top}\tanh(W_1\boldsymbol{x}+\boldsymbol{b}_1), \tag{10}$$

隐藏层 32 个单元，共 704 个参数。设 A 为当前候选集合、$U\subseteq A$ 为尚未被屏蔽的候选，策略为

$$\pi(c\mid s,U)=\frac{\exp f(\boldsymbol{x}_c)}{\sum_{c'\in U}\exp f(\boldsymbol{x}_{c'})},\qquad c\in U. \tag{11}$$

表 2: 候选特征（20 维）

类别	特征
候选本身	是否原地留下、是否送回 ℓ、空驶时间、载货行驶时间、到达后等机器的时间、加工时间、预计完工时刻、预计完工时刻在候选中的排名
工件	剩余最小工作量、剩余工序数、已等待时间
目的工位	占用率、本步后是否占满、机器是否被占着、输入缓冲排队数
出发工位与系统	出发机器的排队数、空闲车辆比例、已完工比例、是否阻塞、缓冲是否有限

死锁规避掩码。 从 $U = A$ 开始，按 π 选出候选（贪心时取概率最大者，采样时随机抽取），用前瞻推演检查；不安全就把它移出 U 并重新归一化再选；U 为空则等待。这样采样得到的动作恰好服从 π 限制在安全候选上的分布，而且只检查被选中的候选，比先检查全部候选省时。策略梯度在接受该动作时的 U 上计算，被屏蔽的候选不接收梯度。缓冲无限时不做检查。

训练。 分两个阶段：

1. **模仿**：对每个训练实例，记录逐例最好规则的每一步决策，用交叉熵训练打分函数。
2. **REINFORCE**：对每个实例采样 K 条轨迹，基线取这 K 次的平均完工时间 b，第 k 条轨迹的优势为 $A_k = (b - C_{\max}^{(k)})/b$，按

$$\nabla_\theta J \approx \frac{1}{K} \sum_{k=1}^{K} A_k \sum_t \nabla_\theta \log \pi_\theta\big(c_t^{(k)} \mid s_t^{(k)}, U_t^{(k)}\big) \tag{12}$$

用 Adam 更新 [18, 19, 20]。

只用验证集选模型，指标为“贪心 $C_{\max}$ / 逐例最好规则”的平均值。

推理。 贪心推理构造一次调度；采样推理在贪心解之外再采样 N 次（本文 $N = 8, 32$），取最好者。学习到的策略本身不保证可行性，可行性由仿真器、死锁规避与检查器共同保证。

4.5 对照方法

模拟退火（SA）。 编码分四段：工序顺序、机器分配、运输顺序、车辆分配。解码先按给定顺序用拓扑排序求最早时间表，缓冲容量作为前后约束；顺序前后矛盾时，先把运输改为按工序顺序排列再试，仍矛盾则退回优先级解码（仿真加死锁规避）。邻域为工序或运输顺序的交换与插入，以及机器或车辆的重新分配。SA 从 4 条规避规则的最好解出发，温度按起始解的比例几何下降，所以结果不劣于规避规则。在同样的编码上我们也实现了遗传算法（GA），在小算例上总体不优于 SA，下文以 SA 为主要参考。

解耦优化。 第一阶段只排机器：D0 不计运输；D1 把载货行驶时间当作工序间隔，假设车辆随时可用。第二阶段固定机器方案（各机器的工序及顺序），用 CP-SAT 只优化车辆与时间。机器方案无法执行时如实记录，不做修补。

5 实验

5.1 算例、配置与实现

- **小算例**：Deroussi 与 Norre[3] 的 10 个 FJSPT 算例（fjsp1–10，5–8 个工件，8 台机器）及其布局，参考解来自 Berterottière 等 [5] 公开的数据。
- **大算例**：Brandimarte[6] 的 mk01–05、mk07（10–20 个工件，55–150 道工序，4–8 台机器），行驶时间取 Homayouni 与 Fontes[4] 的 4/5/6/8 机整数布局。mk06、mk08–10 所需的 10 机、15 机布局含小数时间，未纳入。
- **配置**：基准约定（不阻塞、缓冲无限），以及“阻塞 + 输入缓冲 $B = \infty/2/1/0$”，共 5 种；都是 2 台 AGV、不送回 ℓ。大算例的车辆数是我们设定的，数值不能与原文献直接比较。

- **实现**：Python 3.11、NumPy 2.4、OR-Tools 9.15。CP-SAT 每例 60 s、8 线程；学习型策略只用 CPU。

5.2 基准对齐与参考解

按基准约定，10 个公开参考解重放后 $C_{\max}$ 全部等于作者报告值，并都通过检查器；若改为阻塞，或把装卸站放在最后一个地点，则不一致。这说明本文的模型语义与 FJSPT 基准一致。CP-SAT 在 fjsp1–6、8–10 上证得与参考值相同的最优值（134、114、120、114、94、138、178、144、174）。在 fjsp7 上，CP-SAT 于限时内找到 108（下界 104）；同一算例在“阻塞 + 无限缓冲”配置下证得最优值为 108，而阻塞配置的可行解在基准约定下同样可行，所以基准约定下的最优值在 $[104, 108]$ 内，优于公开参考值 112。

5.3 阻塞与缓冲容量的影响

表 3 给出各配置相对基准约定的 $C_{\max}$ 平均增幅。小算例上用 CP-SAT，未证得最优时用解与下界给出区间；大算例上 CP-SAT 在 60 s 内找不到解（从 55 道工序起，阻塞并限制缓冲后即如此），改用最好的规避规则和 SA。

表 3: 相对基准约定的 $C_{\max}$ 平均增幅

	阻塞 + $B = \infty$	阻塞 + $B = 2$	阻塞 + $B = 1$	阻塞 + $B = 0$
小算例，CP-SAT（证得最优数）	0.21%–0.59%（10）	0.21%–0.59%（10）	−0.16%–1.17%（9）	6.8%–12.8%（5）
大算例，最好规避规则	4.1%	6.0%	11.5%	51.8%
大算例，SA（2 个种子取优）	4.2%	3.0%	9.0%	40.7%

三种方法的定性结论一致：阻塞本身使 $C_{\max}$ 上升约 4%（大算例），缓冲降到 1 时约 10%，无缓冲时大算例上升约 40%–50%。小算例只有 5–8 个工件，缓冲位很少被占满，$B \geq 1$ 时影响不到约 1%，所以缓冲的作用要在更大的算例上研究。更强的方法（SA）把 $B = 0$ 的代价估计从 52% 降到 41%，说明其中一部分来自规则本身太弱。

5.4 联合优化与解耦优化的差距

表 4 在小算例上比较联合 CP-SAT 与两种解耦方法，总预算相同（解耦的两个阶段各 30 s）。可执行的情况下解耦方法的两个阶段都证得最优，所以差距不是限时造成的。

表 4: 协调差距 =（解耦 − 联合）/ 联合，小算例

配置	D0 平均（最大）	D1 平均（最大）	机器方案无法执行
基准约定	10.1%（17.5%）	5.5%（11.9%）	0 / 0
阻塞 + $B = \infty$	11.2%（22.8%）	5.1%（9.3%）	0 / 0
阻塞 + $B = 2$	9.7%（18.5%）	4.8%（11.1%）	0 / 0
阻塞 + $B = 1$	9.8%（22.8%）	4.5%（7.4%）	0 / 0
阻塞 + $B = 0$	20.1%（34.7%）	17.3%（28.8%）	D0 2 / D1 3

在 95 个可执行的情况里，联合 CP-SAT 每一个都严格更好。只要有至少 1 个缓冲位，解耦代价大致保持在 D1 约 5%、D0 约 10%；阻塞且无缓冲时代价升到 17%–20%，还有 2–3 个算例的机器方案根本无法执行，而联合 CP-SAT 在这些算例上都找到了可行调度。即使第一阶段计入了运输时间（D1），由于不考虑阻塞，也照样会排出无法执行的方案。联合 CP-SAT 在少数配置下只有限时可行解，此时真实差距只会更大，表中数字偏保守。

值得注意的是，简单的联合贪心规则 EFT 在小算例上并不比解耦规则好（无缓冲时与逐例最好解耦规则相比 0 胜 10 负），说明联合决策的价值要靠更强的优化能力才能发挥，这也是本文引入学习型方法的动机。

5.5 大算例上的死锁与规避

表 5 给出普通（不加规避）规则在大算例上的死锁次数。$B \le 2$ 时死锁已经很常见，$B = 0$ 时几乎全部死锁，所以在这类问题上，不处理死锁的非延迟规则不能作为基线。加上前瞻规避后，420 次运行（6 算例 $\times$ 5 配置 $\times$ 14 个规则或随机种子）全部给出合法调度，单次最长 1.2 s；不会死锁的情况下，结果与原规则完全相同。

表 5: 普通规则在大算例上的死锁次数 / 运行次数（RANDOM 为 10 个种子）

配置	EFT	FIFO+NV	SPT+NV	MWKR+NV	RANDOM
基准约定、阻塞 + $B = \infty$	0/6	0/6	0/6	0/6	0/60
阻塞 + $B = 2$	1/6	2/6	2/6	2/6	26/60
阻塞 + $B = 1$	3/6	5/6	3/6	6/6	45/60
阻塞 + $B = 0$	6/6	6/6	5/6	6/6	60/60

5.6 学习型策略的评价

训练。 训练集与验证集由生成器产生，种子区间互不重叠：5–10 个工件，每个工件 2–5 道工序，每道工序 1–2 台可选机器，加工时间 5–30；机器数 4/5/6/8，使用上述布局；2 台 AGV，5 种配置随机抽取。训练集 400 个、验证集 100 个。模仿 15 轮（学习率 3×10^{-3}），REINFORCE 120 轮（$K = 8$，每批 8 个实例，学习率 10^{-3}）。3 个训练种子（0、1、2）依次训练，每个种子用 18 个进程并行采样，各用时 29–34 分钟。选中的检查点分别来自第 58、119、74 轮 REINFORCE，验证集上“贪心 / 逐例最好规则”为 1.001、1.002、0.998；只做模仿时验证集上最好为 1.023–1.028。验证集同时用于选模型，这两组数字偏乐观，只作描述。

测试协议。 训练前固定以下设计并登记假设。测试集 T1 为小算例 10 个 $\times$ 5 配置（50 个组合），T2 为大算例 6 个 $\times$ 5 配置（30 个组合）；测试集只在模型选定后使用一次。每个组合上运行：4 条规避规则；每个种子的贪心、采样 8 次、采样 32 次；同等评估次数的 SA（33 次）；同等时间的 SA（时间上限取同一组合上采样 32 次的实测耗时）。推理与同等时间 SA 在同一进程中顺序运行。另引用已归档的大预算 SA（2 个种子取优，T1 每次 5000 次评估、T2 每次 1000 次评估）和 CP-SAT 结果作参照。假设采用单侧配对 Wilcoxon 符号秩检验，在每个种子内对 H1–H4 做 Holm 校正，3 个种子中至少 2 个满足即成立。

假设检验。 表 6 给出结果。4 个假设在 3 个种子上全部成立。H4 虽然显著，幅度只有约 2%。

表 6: 预先登记的假设（胜/平/负按组合计数）

假设	内容	种子 0	种子 1	种子 2	Holm 校正 p
H1	贪心优于 EFT（T1∪T2，$n = 80$）	58/2/20	56/4/20	61/2/17	$\leq 3.3 \times 10^{-6}$
H2	采样 32 优于逐例最好规则（$n = 80$）	77/3/0	76/0/4	79/0/1	$\leq 4.9 \times 10^{-14}$
H3	同等时间优于 SA（T2，$n = 30$）	28/0/2	28/0/2	27/0/3	$\leq 5.8 \times 10^{-6}$
H4	同等时间优于 SA（T1，$n = 50$）	32/4/14	31/4/15	31/5/14	$\leq 5.7 \times 10^{-3}$

描述性结果。 表 7 给出按配置的 $C_{\max}$ 比值（3 个种子平均，小于 1 表示策略更好）。

表 7: 学习型策略的 $C_{\max}$ 比值（3 个种子平均）

测试集	配置	贪心 / 最好规则	采样 32 / 最好规则	同时 SA / 最好规则	采样 32 / SA 参考	采样 32 / 最优（n）
T1	基准约定	0.978	0.928	0.960	1.041	1.115（9）
	$B = \infty$	1.000	0.937	0.959	1.032	1.135（10）
	$B = 2$	1.008	0.925	0.956	1.015	1.120（10）
	$B = 1$	0.999	0.921	0.958	1.028	1.122（9）
	$B = 0$	1.042	0.947	0.945	1.054	1.140（5）
	全部	1.005	0.932	0.956	1.034	1.125（43）
T2	基准约定	0.843	0.797	0.981	0.833	–
	$B = \infty$	0.891	0.818	0.981	0.865	–
	$B = 2$	0.914	0.814	0.980	0.880	–
	$B = 1$	0.925	0.844	0.971	0.902	–
	$B = 0$	1.079	0.911	0.935	1.044	–
	全部	0.931	0.837	0.970	0.905	–

- **大算例上优势最明显。**T2 的工件数和工序数都比训练算例大得多。贪心推理一次构造就比逐例最好规则好 6.9%，与大预算 SA 持平（比值 1.010）；采样 32 次比逐例最好规则好 16.3%，比大预算 SA 好 9.5%，3 个种子分别在 26、24、25/30 个组合上更好。这说明策略学到的打分方式能推广到更大的问题上。
- **小算例上搜索方法仍然更强。**T1 上采样 32 次比大预算 SA 差 3.4%，只在 5–7/50 个组合上更好；距 CP-SAT 证得最优平均 12.5%（贪心 20.9%），43 个证得最优的组合中没有一个达到最优值。作为比较，大预算 SA 距最优约 8%–12%。
- **$B = 0$ 最弱（探索性）。**无缓冲时贪心推理在两组测试集上都不如逐例最好规则，T2 上采样 32 次后仍比大预算 SA 差 4.4%；其余 4 种配置下，T2 的采样 32 次都优于大预算 SA（比值 0.83–0.90）。这与试训时的观察一致。
- **种子间差异很小。**3 个种子的胜负数和比值都很接近，结论不依赖某一个种子。

计算时间。 表 8 为单进程顺序运行的实测时间。大算例上有限缓冲配置的采样较慢：缓冲无限时不做前瞻推演，采样 32 次不到 1 s，所以多出的时间来自推演。

表 8: 推理时间（秒，单进程）

测试集	贪心平均	采样 32 平均	采样 32 最长
T1	0.006	0.20	0.43
T2（全部配置）	0.34	10.9	58.6
T2（$B = 2/1/0$）	–	16.1 / 17.1 / 19.8	–

5.7 讨论

1. **缓冲是关键参数。**三种方法一致表明无缓冲加阻塞使完工时间大幅上升，而一两个缓冲位就能消除大部分损失；对车间设计而言，这提示在机器前保留少量等待位的价值。
2. **联合决策的价值要靠优化能力兑现。**联合 CP-SAT 严格优于解耦优化，但联合贪心规则并不优于解耦规则。学习型策略在同一联合动作空间中学到了比规则更好的打分，尤其在大算例上。
3. **学习型方法的适用范围。**它在大规模、短时间的情形下最有优势；在小算例或允许长时间搜索时，SA 和精确方法更好。这与毕设中维护调度问题上的观察一致。
4. **无缓冲是难点。**$B = 0$ 时前瞻规避最频繁地介入，可选动作少，策略与规则的差别也最小；针对这一配置的特征或训练比重可能是改进方向。

6 结论与展望

本文研究带 AGV 运输、有限输入缓冲与加工后阻塞的柔性作业车间调度。在与公开基准完全对齐的模型上，我们定量给出了阻塞与缓冲容量的影响，以及联合与解耦决策的差距随缓冲变紧的变化；提出了以保守推演为基础的死锁规避，并把它作为学习型构造策略的动作掩码。在预先登记的检验中，学习型策略显著优于规则与同等时间的模拟退火，在比训练算例更大的算例上优于大预算模拟退火。

局限。

- 测试算例少（小算例 10 个、大算例 6 个），同一算例的 5 种配置之间不独立。
- 大算例没有精确参考，“大预算 SA”每次也只有 1000 次评估。
- “同等时间”的比较依赖两种 Python 实现的效率，换实现结论可能变化。
- 只考虑 2 台同质单载 AGV、静态问题，不含路径冲突与充电；死锁规避的完备性没有严格证明。
- 没有与文献中的深度强化学习方法直接比较，也没有做消融实验。

展望。 用学习型策略的解作为搜索的初始解，以弥补小算例上的差距；加快前瞻推演；扩展到新工件到达、机器故障等动态事件，以及路径冲突与异构车辆。

参考文献

[1] Dauzère-Pérès S, Ding J, Shen L, Tamssaouet K. The flexible job shop scheduling problem: a review. *European Journal of Operational Research*, 2024, 314(2): 409–432.

[2] Bilge Ü, Ulusoy G. A time window approach to simultaneous scheduling of machines and material handling system in an FMS. *Operations Research*, 1995, 43(6): 1058–1070.

[3] Deroussi L, Norre S. Simultaneous scheduling of machines and vehicles for the flexible job shop problem. International Conference on Metaheuristics and Nature Inspired Computing, 2010. [待核对：会议名称与页码]

[4] Homayouni S M, Fontes D B M M. Production and transport scheduling in flexible job shop manufacturing systems. *Journal of Global Optimization*, 2021, 79(2): 463–502.

[5] Berterottière L, Dauzère-Pérès S, Yugma C. Flexible job-shop scheduling with transportation resources. *European Journal of Operational Research*, 2024, 312(3): 890–909.

[6] Brandimarte P. Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 1993, 41: 157–183.

[7] Zhuang C, et al. Comprehensive MILP formulation and solution for simultaneous scheduling of machines and AGVs in a partitioned flexible manufacturing system. *Machines*, 2025, 13: 519. [待核对：作者全名]

[8] Moinuddin A, Er Raqabi E M, Van Hentenryck P. Scheduling and routing in the flexible job shop with heterogeneous transbots and zoning: a constraint programming approach. arXiv:2604.24483, 2026.

[9] Link M, Hoss J, Klarmann N. An analysis of the coordination gap between joint and modular learning for job shop scheduling with transportation resources. IEEE CASE 2026; arXiv:2604.24117.

[10] A heuristic-assisted deep reinforcement learning algorithm for flexible job shop scheduling with transport constraints. *Complex & Intelligent Systems*, 2025. doi:10.1007/s40747-025-01828-6. [待核对：作者]

[11] Wang J, et al. Dynamic integrated scheduling of production equipment and automated guided vehicles in a flexible job shop based on deep reinforcement learning. *Processes*, 2024, 12(11): 2423. [待核对：作者全名]

[12] Hu L, et al. Petri-net-based dynamic scheduling of flexible manufacturing system via deep reinforcement learning with graph convolutional network. *Journal of Manufacturing Systems*, 2020, 55. doi:10.1016/j.jmsy.2020.02.004. [待核对：题目、作者与页码]

[13] Zhang S, et al. arXiv:2602.24180, 2026. [待补全：题目与作者；内容为带有限缓冲与配料约束的柔性作业车间强化学习调度]

[14] 王东海, 等. 制造技术与机床, 2026. [待补全：按知网补全题目与卷期；内容为带中央缓存的重入阻塞流水车间深度强化学习调度]

[15] 余娜娜, 等. 中国机械工程, 2026. [待补全：按知网补全题目与卷期；内容为阻塞与异构 **AGV** 下的混合流水车间调度]

[16] Zhang C, Song W, Cao Z, Zhang J, Tan P S, Xu C. Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in Neural Information Processing Systems*, 2020.

[17] Song W, Chen X, Li Q, Cao Z. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 2023, 19(2): 1600–1610. [待核对]

[18] Williams R J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992, 8: 229–256.

[19] Kool W, van Hoof H, Welling M. Attention, learn to solve routing problems! International Conference on Learning Representations, 2019.

[20] Kwon Y-D, Choo J, Kim B, Yoon I, Gwon Y, Min S. POMO: policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems*, 2020.

[21] Garey M R, Johnson D S, Sethi R. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1976, 1(2): 117–129.

[22] Perron L, Didier F. CP-SAT. Google OR-Tools, `https://developers.google.com/optimization`.